

PROJET DE SYNTHESE

ELHAM-MOHAMAD-AKRAM-LENAIC



Université de Bretagne Occidentale
Département informatique
Année 2025-2026

1.Introduction

Ce rapport présente le travail réalisé dans le cadre du Projet de Synthèse de Licence 3 Informatique à l'Université de Bretagne Occidentale. L'objectif principal était de concevoir et développer un interpréteur de S-Expressions permettant d'animer des éléments graphiques 2D, puis d'étendre cette application en système distribué client-serveur.

Le projet s'articule en deux grandes parties complémentaires. La première partie consiste en la construction progressive d'un interpréteur de scripts basé sur la syntaxe des S-Expressions, inspirée des langages fonctionnels Lisp et Scheme. La seconde partie vise à transformer cet interpréteur en une application distribuée dotée d'une interface graphique ergonomique, d'un mécanisme de persistance des données, d'une fonctionnalité de capture d'écran et d'éléments graphiques animés autonomes appelés Robibots.

L'ensemble du développement a été réalisé en Java sous Eclipse, avec un dépôt Git hébergé sur GitLab et un pipeline d'intégration continue automatisé.

2. Architecture générale du projet

2.1 Organisation du dépôt

Le projet est structuré en trois modules distincts répartis dans le même dépôt Git. Le module 2DGraphicCore fournit la couche graphique 2D de bas niveau, comprenant les classes GSpace, GRect, GOval, GImage et GString. Le module SParser est le parseur de S-Expressions fourni, capable de compiler une chaîne de caractères en un arbre de SNode. Enfin, le module Robi constitue le projet principal regroupant l'ensemble des exercices, du client Swing, du serveur TCP et des Robibots.

2.2 Progression des exercices

Le code est organisé de façon progressive, chaque package correspondant à un exercice qui étend le précédent. L'exercice 1 implémente une animation graphique de base. L'exercice 2 pose les fondations de l'interpréteur. L'exercice 3 introduit le patron Command. L'exercice 4 met en place la structure Environment/Reference. L'exercice 5 gère les conteneurs imbriqués. L'exercice 6 ajoute la notion de scripts utilisateur. L'exercice 7 complète le langage avec des expressions, conditionnelles et boucles.

3. Partie 1 – Interpréteur de S-Expressions

(receiver commande argument1 argument2 ...)

3.1 Langage de script

Le langage de script s'appuie sur la syntaxe des S-Expressions. Chaque commande suit la forme générale :

(receiver commande argument1 argument2 ...)

Les commandes disponibles couvrent la manipulation complète des éléments graphiques. La commande `setColor` modifie la couleur d'un élément ou du fond. La commande `translate` déplace un élément d'un décalage en pixels. La commande `setDim` redimensionne un élément. La commande `add` crée et ajoute un nouvel élément dans un conteneur. La commande `del` supprime un élément. La commande `clear` vide intégralement un conteneur. La commande `sleep` introduit une pause en millisecondes.

Exemple complet : `(space add robi (rect.class new))` crée un rectangle nommé `robi` dans le conteneur principal, puis `(robi setColor yellow)` lui applique une couleur jaune.

3.2 Patron de conception Command

Dès l'exercice 3, chaque commande du langage est encapsulée dans une classe Java implémentant l'interface Command. Cette interface définit une unique méthode run prenant en paramètre la référence du receveur, le nœud SNode de la méthode et l'environnement courant.

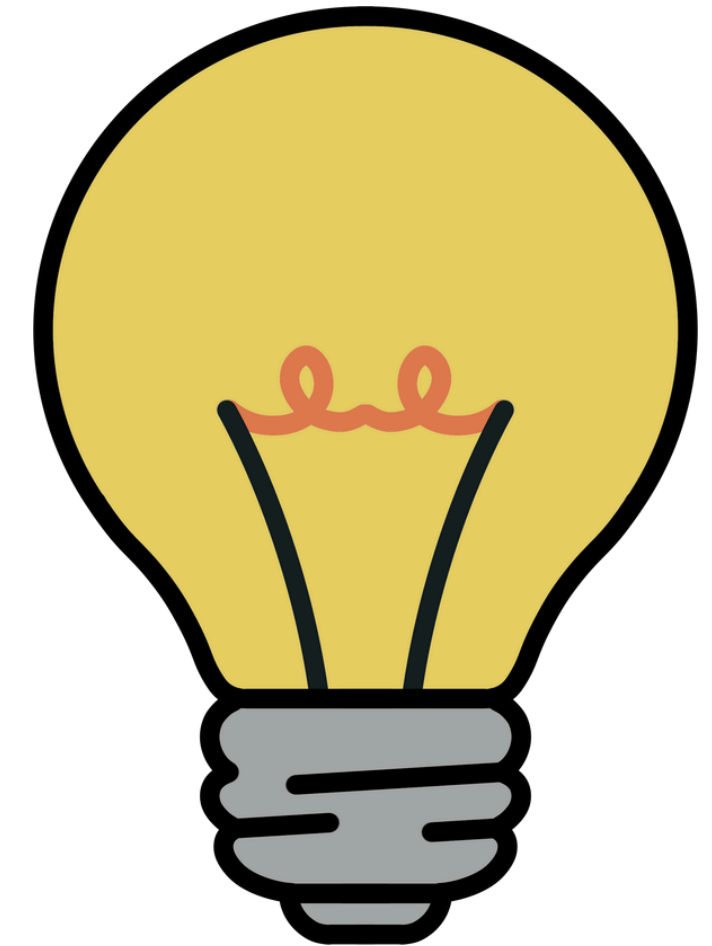
Ce patron offre plusieurs avantages structurels importants. Il garantit l'extensibilité du système : ajouter une nouvelle commande ne nécessite pas de modifier l'interpréteur existant, il suffit de créer une nouvelle classe et de l'enregistrer. Il assure la séparation des responsabilités, chaque classe étant responsable d'une seule commande. Il améliore également la testabilité, chaque commande pouvant être testée de façon indépendante.

3.3 Architecture Environment / Reference

L'exercice 4 introduit une structure en arbre d'objets qui remplace complètement les doubles conditionnelles imbriquées de la version précédente. Cette architecture repose sur trois classes centrales.

La classe Environment est le registre global de l'application. Elle associe un nom textuel à une instance de Reference via une HashMap. La classe Reference encapsule un objet graphique quelconque et maintient un dictionnaire de commandes disponibles pour cet objet. L'interface Command définit le contrat d'exécution que chaque commande doit respecter.

Cette structure permet une sélection de commande entièrement dynamique, sans aucune conditionnelle dans la boucle principale d'interprétation. La méthode run de Reference recherche simplement la commande par son nom dans le dictionnaire et délègue l'exécution.



3.4 Conteneurs imbriqués et notation pointée

L'exercice 5 étend le modèle pour permettre l'imbrication d'éléments graphiques au sein d'autres conteneurs, GRect et GOval étant eux-mêmes des GContainer. La notation pointée, par exemple `space.robi.im`, permet d'accéder sans ambiguïté à un élément dans un conteneur précis, y compris lorsque plusieurs éléments portent le même nom dans des conteneurs différents.

3.5 Scripts utilisateur

L'exercice 6 introduit la notion de script défini par l'utilisateur. Via la commande `addScript`, il est possible d'associer un nouveau comportement paramétré à n'importe quelle référence. Le mot-clé `self` joue le rôle de `this` en Java, référençant l'objet receveur du script. La substitution des paramètres est réalisée récursivement sur l'arbre de `SNode` avant l'exécution du corps du script.

Exemple : `(space addScript addImage ((self filename) (self add im (Image new filename))))`
définit une commande `addImage` réutilisable sur `space`.

3.6 Expressions, conditionnelles et boucles

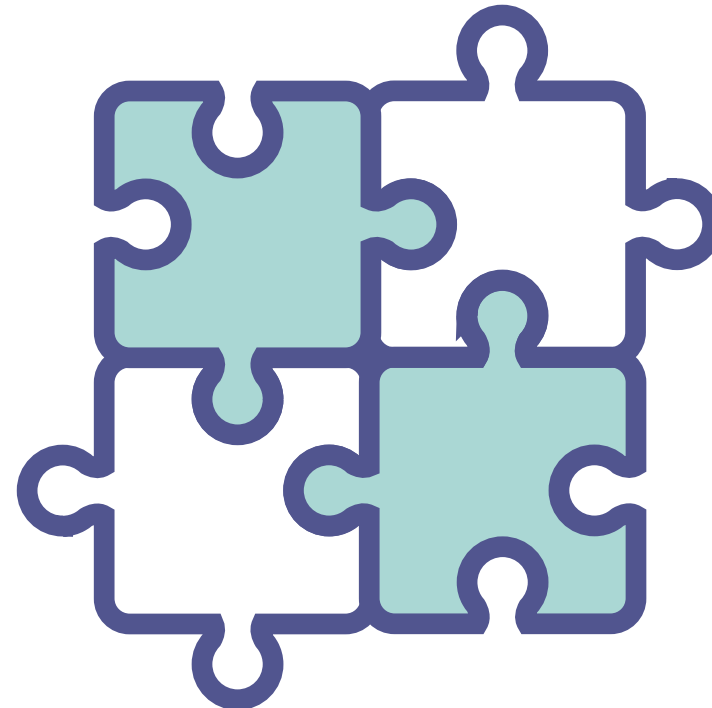
L'exercice 7 complète le langage de script en ajoutant des constructions de contrôle inspirées de Scheme. Les conditionnelles et les boucles permettent désormais d'écrire des scripts comportant une logique de contrôle de flux, rapprochant le langage d'un interpréteur fonctionnel à part entière.

4. PARTIE 2 – APPLICATION DISTRIBUÉE

client-serveur

4.1 Architecture client-serveur

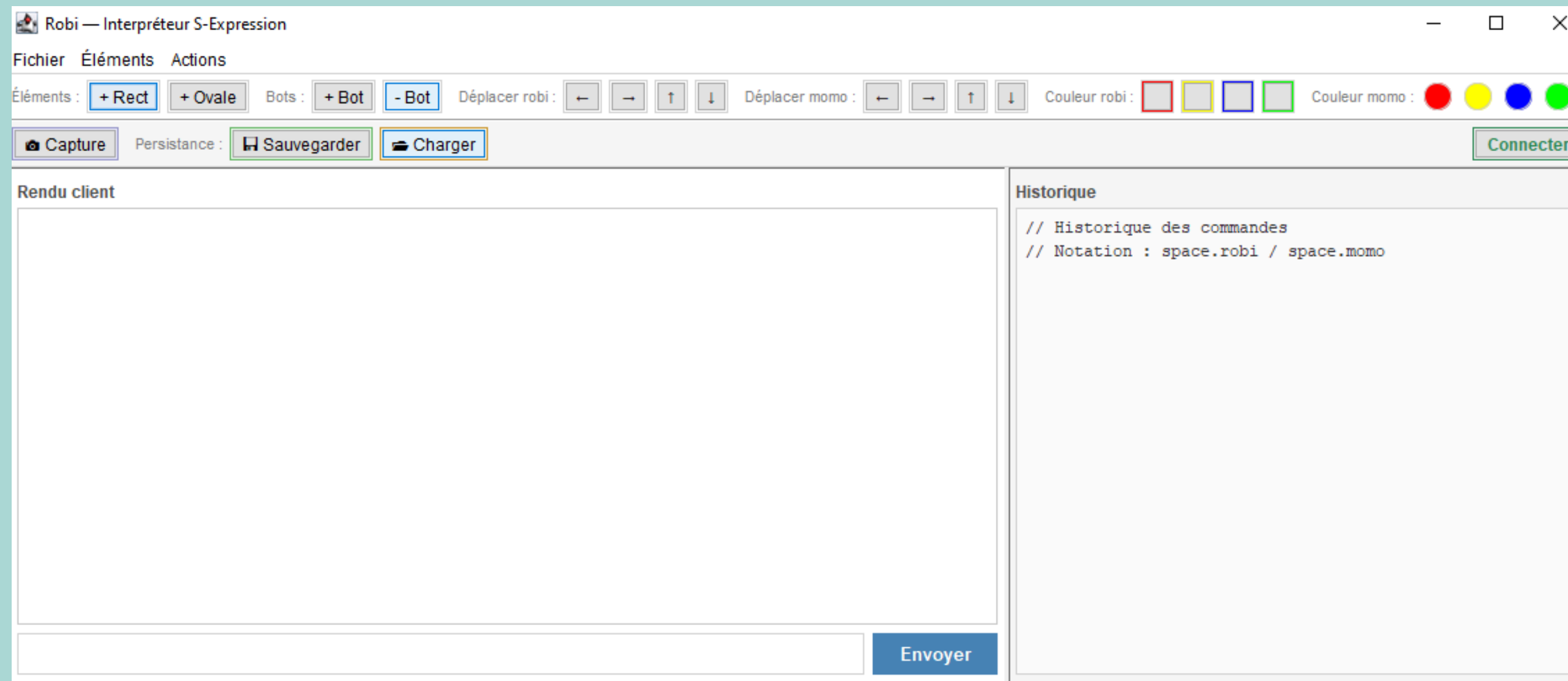
L'application adopte une architecture TCP classique. Le serveur écoute sur le port 4242 et crée un thread dédié par client connecté. Un unique interpréteur RobiInterpreter est partagé entre tous les clients : il exécute les commandes reçues et maintient le rendu graphique côté serveur via un GSpace AWT.



Le protocole d'échange est le suivant : le client envoie une S-Expression en texte terminée par un saut de ligne. Le serveur parse la commande, l'exécute et répond avec un objet Java sérialisé. Selon la nature de la requête, la réponse est une liste de SNode pour les commandes normales, un tableau d'octets PNG pour les captures d'écran, ou une chaîne JSON pour la sauvegarde de l'état.

La classe ServerConnection côté client gère deux canaux de communication distincts : un canal synchrone pour les commandes normales, et un thread d'écoute asynchrone pour recevoir les mises à jour des Robibots diffusées par le serveur.

4.2 INTERFACE GRAPHIQUE



L'INTERFACE GRAPHIQUE A ÉTÉ CONÇUE EN RESPECTANT LES BONNES PRATIQUES D'ERGONOMIE ET D'UTILISABILITÉ. ELLE SE COMPOSE D'UNE BARRE D'OUTILS OFFRANT UN ACCÈS RAPIDE AUX ACTIONS LES PLUS FRÉQUENTES : AJOUT DE FORMES GÉOMÉTRIQUES, GESTION DES ROBIBOTS, DÉPLACEMENT DIRECTIONNEL, SÉLECTION DE COULEUR, VIDAGE DU CANVAS, CAPTURE D'ÉCRAN ET PERSISTANCE. UN PANNEAU DE RENDU CLIENT AFFICHE EN TEMPS RÉEL LE RÉSULTAT DES COMMANDES EXÉCUTÉES CÔTÉ CLIENT. UN HISTORIQUE DES COMMANDES PERMET DE TRACER L'ENSEMBLE DES S-EXPRESSIONS ENVOYÉES AU SERVEUR. ENFIN, UN CHAMP DE SAISIE LIBRE PERMET À L'UTILISATEUR EXPÉRIMENTÉ D'ENVOYER DIRECTEMENT DES S-EXPRESSIONS PERSONNALISÉES.

4.3 CAPTURE D'ÉCRAN

La fonctionnalité de capture est déclenchée par le bouton dédié ou la commande (space screenshot). Le serveur capture le contenu de son GSpace en mémoire sous forme d'image PNG, le sérialise en tableau d'octets et le transmet au client. Celui-ci reçoit l'image et l'affiche dans une fenêtre dédiée. Cette fonctionnalité permet de comparer visuellement le rendu côté serveur et le rendu côté client, conformément aux exigences du sujet.

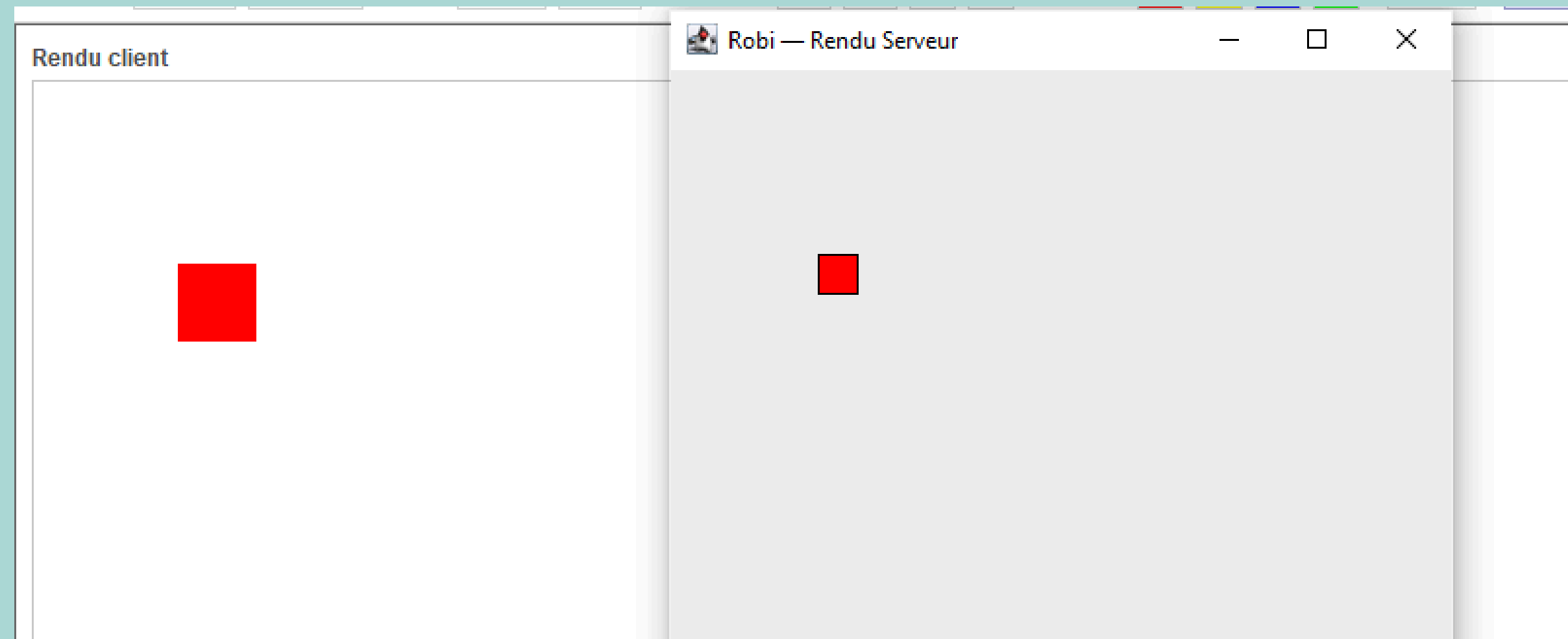
4.4 PERSISTANCE

Le mécanisme de persistance permet de sauvegarder et de recharger l'état complet du dessin entre deux sessions. Le format retenu est JSON, choisi pour sa lisibilité et sa portabilité. Le bouton Sauvegarder envoie la commande (space save) au serveur, qui sérialise l'environnement courant en JSON et le retourne au client pour une sauvegarde locale. Le bouton Charger relit ce fichier JSON et réenvoie séquentiellement les commandes au serveur pour reconstituer fidèlement l'état précédent.

4.5 ROBIBOTS – MACHINE À ÉTATS

Les Robibots sont des éléments graphiques animés dont le comportement est modélisé par une machine à états finie, implémentée from scratch sans librairie externe, conformément aux contraintes du sujet. Chaque Robibot possède un état courant, des transitions déclenchées par des événements tels que le contact avec les bords ou la collision avec un autre bot, et des actions associées à chaque état.

La synchronisation des Robibots est centralisée côté serveur. L'interpréteur maintient un tick régulier qui calcule les nouveaux états de chaque bot, met à jour le rendu serveur et diffuse les commandes de déplacement à l'ensemble des clients connectés via le mécanisme d'observateurs BotUpdateListener. Cette approche garantit la cohérence de l'état global entre tous les clients.

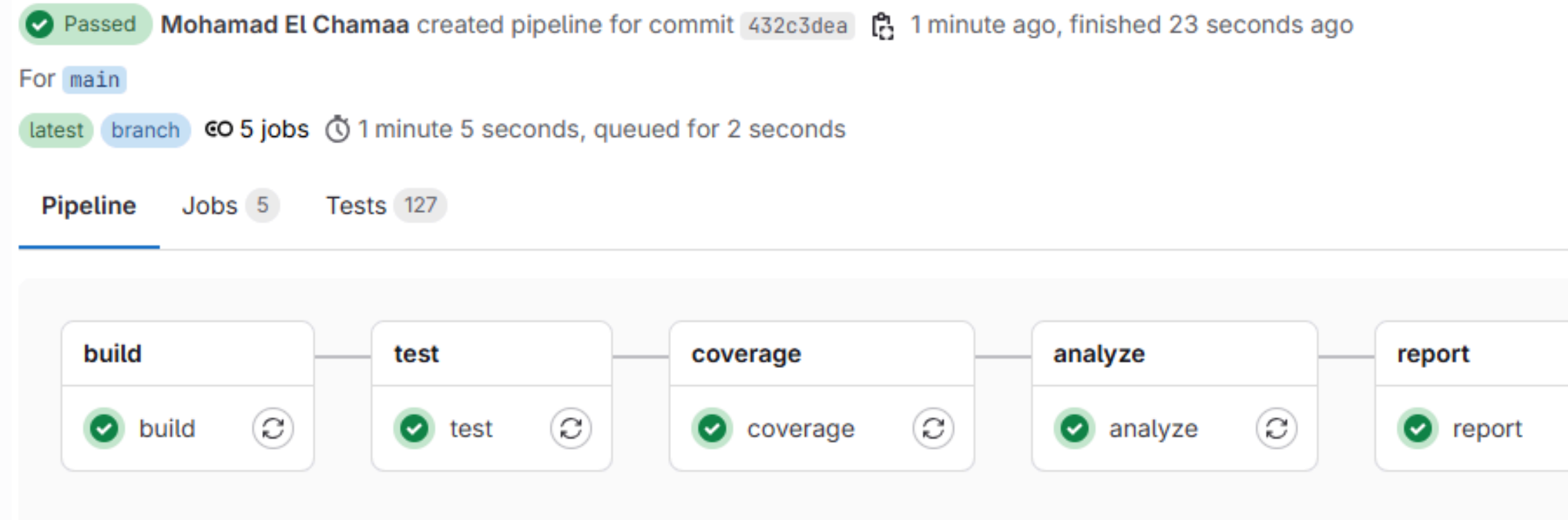


5. QUALITÉ DU CODE ET PIPELINE CI/CD

Un fichier `.gitlab-ci.yml` a été mis en place pour automatiser la chaîne qualité à chaque push ou merge request sur le dépôt. Le pipeline comprend cinq étapes successives : la compilation du projet via Maven, l'exécution des tests unitaires JUnit, la mesure de la couverture de code avec JaCoCo, l'analyse statique avec Checkstyle et SpotBugs pour détecter les mauvaises pratiques, et enfin la génération d'un rapport qualité consolidé.

Les branches du dépôt sont organisées selon une stratégie classique : `main` pour le code stable, `release` pour les versions livrables, `dev` pour l'intégration continue des développements, et des branches `feat` pour chaque fonctionnalité en cours.

fix :



5.2 POINTS FORTS

PLUSIEURS ÉLÉMENTS POSITIFS RESSORTENT DE L'ANALYSE DU PROJET.

- **L'ARCHITECTURE ORIENTÉE OBJET EST CLAIRE ET PROGRESSIVE. CHAQUE EXERCICE ÉTEND LE PRÉCÉDENT SANS RUPTURE DE CONCEPTION, FORMANT UN TOUT COHÉRENT.**
- **LE PATRON COMMAND EST CORRECTEMENT APPLIQUÉ ET GARANTIT L'EXTENSIBILITÉ DU SYSTÈME SANS MODIFICATION DE L'INTERPRÉTEUR EXISTANT.**
- **LA SÉPARATION ENTRE LA COUCHE GRAPHIQUE, L'INTERPRÉTEUR ET LA COUCHE RÉSEAU EST NETTE ET BIEN RESPECTÉE.**
- **L'INTERFACE SWING EST ERGONOMIQUE ET OFFRE UN RETOUR VISUEL IMMÉDIAT À L'UTILISATEUR.**
- **LA MACHINE À ÉTATS DES ROBIBOTS EST IMPLÉMENTÉE FROM SCRATCH, CONFORMÉMENT AUX CONTRAINTES IMPOSÉES PAR LE SUJET.**
- **LE PIPELINE CI/CD EST OPÉRATIONNEL ET COUVRE LES ÉTAPES ESSENTIELLES DE LA CHAÎNE QUALITÉ.**



Université de Bretagne Occidentale

MERCI POUR VOTRE ATTENTION

***TRAVAIL PRÉSENTÉ ET RÉDIGÉ PAR :
ELHAM JABER-
AKRAM ALNEZAMI -
MOHAMAD ELCHAMAA-
LENAIC HOUNLEBA***

***ET ENCADRÉ PAR :
ALAIN PLANTEC
YANN GLÉMAREC***